# Cours Magistral Complet

Apprentissage par Renforcement (Reinforcement Learning)

M1 Intelligence Artificielle

2026

# Table des matières

# Chapitre 1

# Introduction au Reinforcement Learning

## 1.1 Le Concept Fondamental

L'Apprentissage par Renforcement (ou *Reinforcement Learning* - RL) est l'un des trois paradigmes majeurs du Machine Learning. Contrairement aux approches classiques où l'on fournit des données corrigées à la machine, le RL s'intéresse à la **prise de décision séquentielle**.

Le principe est inspiré de la psychologie comportementale : une entité (l'**Agent**) apprend à agir dans un monde (l'**Environnement**) en observant les conséquences de ses actions.

— Si l'issue est positive, l'action est renforcée (récompensée).

— Si l'issue est négative, l'action est découragée (punie).

**Intuition & Compréhension : L'Analogie de l'Enfant**

Imaginez un enfant apprenant à marcher. Personne ne lui donne une équation différentielle de la gravité ou ne lui dit "contracte le muscle quadriceps de 12%". L'enfant essaie de se lever, tombe (douleur/échec), essaie de modifier son appui, réussit à tenir debout (joie/succès). C'est exactement ce que fait un agent RL : il apprend par **essais et erreurs** (Trial and Error).

## 1.2 Comparaison avec les Autres Paradigmes

Il est crucial de situer le RL par rapport aux autres méthodes d'IA pour comprendre sa spécificité.

### 1.2.1 Vs Apprentissage Supervisé (Supervised Learning)

En apprentissage supervisé (ex : classification d'images), un "professeur" fournit la réponse correcte pour chaque exemple (Image de chat → Label "Chat"). L'objectif est de minimiser l'erreur entre la prédiction et la vérité terrain.

— **Supervisé** : "Voici la bonne réponse, apprends-la."

— **RL** : "Voici une note sur 20 après que tu aies agi. Débrouille-toi pour comprendre ce qui était bien ou mal."

Le RL est plus difficile car le signal de feedback (la récompense) est souvent **rare** et **retardé**. Si vous jouez aux échecs, vous ne savez si votre premier coup était bon qu'à la toute fin de la partie.

### 1.2.2 Vs Apprentissage Non-Supervisé (Unsupervised Learning)

L'apprentissage non-supervisé cherche des structures cachées dans des données (clustering). Le RL, lui, a un but explicite : **maximiser une récompense**. Il ne cherche pas juste à comprendre les données, il cherche à **contrôler** un processus.

## 1.3 Le Cycle d'Interaction Agent-Environnement

Tout problème de RL se formalise par une boucle de rétroaction infinie entre deux entités.

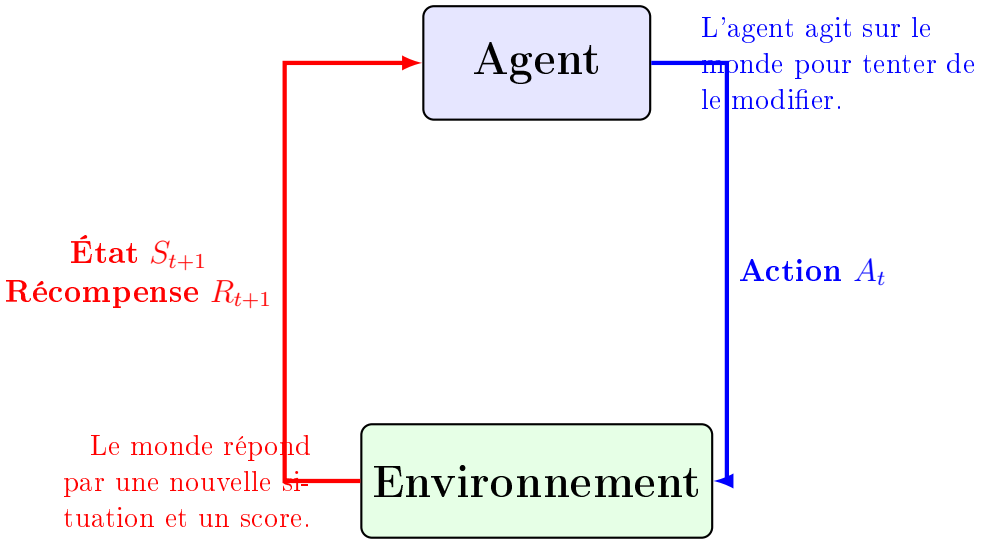


FIGURE 1.1 – La boucle Agent-Environnement. À chaque pas de temps $t$, l'agent reçoit $S_t$, joue $A_t$, et reçoit $R_{t+1}, S_{t+1}$.

L'hypothèse fondamentale est que l'agent n'a pas le contrôle total sur l'environnement (il y a une part d'incertitude), mais il peut l'influencer.

# Chapitre 2

# Formalisation Mathématique : Les MDP

Pour résoudre informatiquement ces problèmes, nous devons définir un cadre mathématique rigoureux. Ce cadre est le **Processus de Décision de Markov** (Markov Decision Process - MDP).

## 2.1 Définition d'un MDP

Un MDP est défini par un tuple de 5 éléments : $(S, A, P, R, \gamma)$. Détaillons chacun d'eux avec précision.

**Formalisme Mathématique : Composants du MDP**

1. $S$ **(State Space)** : L'ensemble de tous les états possibles. C'est la description complète de la situation à un instant $t$. Ex : La position de toutes les pièces sur un échiquier.
2. $A$ **(Action Space)** : L'ensemble de toutes les actions que l'agent peut effectuer. Ex : "Bouger Pion A en C3".
3. $P$ **(Probabilité de Transition)** : La dynamique du monde. C'est la probabilité de passer de l'état $s$ à l'état $s'$ en ayant fait l'action $a$.

   $$P(s'|s, a) = \mathbb{P}(S_{t+1} = s'|S_t = s, A_t = a)$$

   Ceci modélise l'incertitude (le vent qui dévie le robot, l'adversaire aux échecs...).
4. $R$ **(Fonction de Récompense)** : Le feedback immédiat reçu après une action.

   $$R(s, a) = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$$
5. $\gamma$ **(Facteur d'Escompte / Discount Factor)** : Un nombre entre 0 et 1 qui détermine l'importance du futur.

## 2.2 La Propriété de Markov

Pourquoi appelle-t-on cela "Markovien" ? C'est une hypothèse simplificatrice très forte mais nécessaire. Un état est dit **Markov** si "le futur est indépendant du passé sachant le présent".

Autrement dit, pour prendre la bonne décision, l'agent a juste besoin de connaître l'état actuel $S_t$. Il n'a pas besoin de se souvenir de tout l'historique $S_0, S_1, \ldots$. L'état actuel résume toute l'information utile.

$$\mathbb{P}(S_{t+1}|S_t) = \mathbb{P}(S_{t+1}|S_1, S_2, \ldots, S_t)$$

## 2.3 L'Objectif : Le Retour (Return)

Le but de l'agent n'est pas de maximiser la récompense immédiate (sinon il ne ferait jamais d'effort dont la récompense est lointaine, comme étudier pour un examen). Son but est de maximiser la somme cumulée des récompenses sur le long terme.

On appelle cette somme le **Retour** ($G_t$).

**Formalisme Mathématique : Le Retour Escompté (Discounted Return)**

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Le facteur $\gamma$ est crucial :

- Si $\gamma \to 0$ : L'agent est "myope". Il ne s'intéresse qu'à $R_{t+1}$.
- Si $\gamma \to 1$ : L'agent est "prévoyant". Il accorde autant d'importance au futur lointain qu'au présent.

Mathématiquement, $\gamma < 1$ assure aussi que la somme infinie converge vers une valeur finie.

# Chapitre 3

# Politiques et Fonctions de Valeur

C'est le cœur de la théorie du RL. Comment quantifier si une situation est "bonne" ou "mauvaise" ?

## 3.1 La Politique (Policy $\pi$)

La politique est le "cerveau" de l'agent. C'est la fonction qui décide quelle action prendre dans un état donné.

— **Politique Déterministe** : $a = \pi(s)$. Pour un état donné, l'action est toujours la même.

— **Politique Stochastique** : $\pi(a|s) = \mathbb{P}(A_t = a|S_t = s)$. L'agent choisit une action selon une probabilité. (Ex : 80% Avancer, 20% Reculer). C'est crucial pour l'exploration.

L'objectif du RL est de trouver la **Politique Optimale** $\pi^*$ qui maximise le retour espéré.

## 3.2 Les Fonctions de Valeur (Value Functions)

Pour trouver la meilleure politique, l'agent doit être capable d'estimer la "valeur" de chaque état. Il existe deux fonctions fondamentales pour cela.

### 3.2.1 1. La State-Value Function $V^\pi(s)$

La fonction $V^\pi(s)$ répond à la question : **"Si je suis dans l'état $s$ et que je continue avec ma stratégie habituelle $\pi$, combien de points je vais gagner en moyenne jusqu'à la fin ?"**

$$V^\pi(s) = \mathbb{E}_\pi[G_t|S_t = s]$$

### 3.2.2 2. La Action-Value Function $Q^\pi(s, a)$

La fonction $Q^\pi(s, a)$ est plus précise. Elle répond à : **"Si je suis dans l'état $s$, que je force l'action $a$ maintenant (même si ma politique disait autre chose), et qu'ensuite je reprends ma stratégie habituelle $\pi$, combien je gagne ?"**

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

Cette fonction $Q$ est fondamentale pour les algorithmes *Model-Free* (comme le Q-Learning) car elle permet de comparer les actions directement sans connaître la dynamique de l'environnement.

## 3.3 L'Équation de Bellman

Richard Bellman (1957) a formulé une récursivité qui est la clé de tout le RL. L'idée est simple : La valeur d'un état est égale à la récompense immédiate plus la valeur de l'état suivant (pondérée par $\gamma$).

**Formalisme Mathématique : Équation de Bellman pour $V^\pi$**

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s'|s,a) \left[r + \gamma V^\pi(s')\right]$$

**Interprétation pas à pas :**

1. $\sum_a \pi(a|s)$ : On fait la moyenne sur toutes les actions que je pourrais choisir.
2. $\sum_{s'} P(s'|s,a)$ : On fait la moyenne sur tous les états où cette action pourrait m'amener (car l'environnement est incertain).
3. $r + \gamma V^\pi(s')$ : C'est le score total = mon gain immédiat + la valeur de ma future destination.

Cela nous permet aussi de lier $V$ et $Q$ :

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) Q^\pi(s,a)$$

# Chapitre 4

# Résolution "Model-Based" (Programmation Dynamique)

Si l'agent connaît parfaitement le modèle du monde (il connaît les fonctions $P$ et $R$), le problème est "résolu" théoriquement. Il n'a pas besoin d'explorer, il peut juste *calculer* la solution. On appelle cela la **Programmation Dynamique**.

Il existe deux algorithmes principaux.

## 4.1 Algorithme 1 : Policy Iteration

Le principe est d'alterner entre deux phases jusqu'à converger vers la politique optimale.

1. **Policy Evaluation (Évaluation)** : On fixe la politique $\pi$. On calcule sa valeur $V^\pi(s)$ précise pour tous les états (en résolvant le système d'équations de Bellman).
2. **Policy Improvement (Amélioration)** : On regarde les valeurs calculées. Pour chaque état, on change la politique pour choisir l'action qui donne le meilleur résultat immédiat (Greedy) selon $V^\pi$. $\pi_{new}(s) = \arg\max_a Q^\pi(s, a)$.

On boucle : Évaluer $\rightarrow$ Améliorer $\rightarrow$ Évaluer $\rightarrow$ Améliorer... Jusqu'à ce que la politique ne change plus.

## 4.2 Algorithme 2 : Value Iteration

Parfois, évaluer parfaitement la politique (étape 1) est trop long. L'algorithme *Value Iteration* raccourcit le processus en combinant les deux étapes. Au lieu d'attendre l'évaluation complète, on met à jour la valeur $V(s)$ en utilisant directement l'équation d'optimalité de Bellman (avec le max) à chaque étape.

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} P(s'|s,a)[r + \gamma V_k(s')]$$

C'est souvent plus rapide et plus simple à implémenter.

# Chapitre 5

# Résolution "Model-Free" (Apprentissage Réel)

Dans la vraie vie, on ne connaît pas les probabilités de transition $P$. On ne peut pas calculer d'espérances. On doit **apprendre en agissant**. C'est le domaine du **Model-Free RL**.

## 5.1 Monte Carlo vs Temporal Difference (TD)

Il y a deux façons d'apprendre sans modèle :

**Monte Carlo (MC)** L'agent joue un épisode complet jusqu'à la fin (Game Over). Il regarde le score total obtenu ($G_t$). Il met à jour les états visités en disant "Tiens, cet état m'a mené à une victoire, je vais augmenter sa valeur".

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

*Problème : Il faut attendre la fin de l'épisode (impossible pour les tâches continues) et la variance est élevée.*

**Temporal Difference (TD)** L'agent n'attend pas la fin. Il fait un pas, reçoit une récompense $R_{t+1}$, se retrouve en $S_{t+1}$, et utilise sa propre estimation actuelle de $V(S_{t+1})$ pour mettre à jour $V(S_t)$. C'est le principe du **Bootstrapping** (apprendre d'une estimation).

$$V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{Cible TD}} - V(S_t)]$$

*Avantage : Apprentissage pas-à-pas, très efficace.*

## 5.2 Algorithmes de Contrôle (Q-Learning et SARSA)

Le but est d'apprendre les $Q(s, a)$ pour déduire la meilleure action $\max_a Q(s, a)$.

### 5.2.1 SARSA (On-Policy)

SARSA pour (State-Action-Reward-State-Action). L'agent apprend la valeur de la politique qu'il est en train de suivre *réellement*, y compris ses erreurs d'exploration.

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$$

L'action $A'$ est celle que l'agent va vraiment faire au tour suivant.

### 5.2.2 Q-Learning (Off-Policy)

C'est l'algorithme le plus célèbre. L'agent apprend la valeur de la politique **optimale**, peu importe ce qu'il fait dans la réalité. Pour la mise à jour, il ne regarde pas l'action $A'$ qu'il va faire, mais la **meilleure action possible** $\max_{a'} Q(S', a')$.

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)]$$

C'est très puissant car même si l'agent explore au hasard, il apprend la stratégie parfaite en arrière-plan.

# Chapitre 6

# Le Dilemme Exploration-Exploitation

C'est défi fondamental du RL. En arrivant dans un restaurant :

— **Exploitation** : Vous commandez votre plat préféré. Vous êtes sûr d'avoir un bon repas (maximisation immédiate), mais vous ne découvrirez jamais ce qu'il y a d'autre.

— **Exploration** : Vous commandez un plat au hasard. Vous risquez d'être déçu (reward négatif), mais vous pourriez découvrir un plat encore meilleur (découverte de l'optimum global).

Si on ne fait que l'Exploitation, on reste bloqué dans une solution sub-optimale. Si on ne fait que l'Exploration, on ne gagne jamais de points.

## 6.1 La Solution : Epsilon-Greedy

La méthode standard est d'utiliser un paramètre $\epsilon$ (epsilon).

— Avec une probabilité $\epsilon$, l'agent choisit une action totalement **aléatoire**.

— Avec une probabilité $1 - \epsilon$, l'agent choisit l'action qu'il pense être la meilleure (**greedy**).

Généralement, on utilise un **Epsilon Decay** : on commence avec $\epsilon = 1$ (exploration totale) au début de l'apprentissage, et on le diminue progressivement vers 0.01 pour laisser place à l'exploitation une fois que l'agent est entraîné.

# Chapitre 7

# Mise en Pratique : Code Commenté

Voici une implémentation complète de Q-Learning sur l'environnement "FrozenLake" de la librairie Gymnasium.

```
import gymnasium as gym
import numpy as np

# 1. Cr ation de l'environnement
# FrozenLake : Une grille glissante. But : aller de S    G sans tomber dans les trous H.
# is_slippery=True rend l'environnement stochastique (transitions incertaines)
env = gym.make("FrozenLake-v1", is_slippery=True)

# 2. Initialisation de la Q-Table
# Une matrice de taille [Nb_Etats, Nb_Actions] initialis e    0
q_table = np.zeros((env.observation_space.n, env.action_space.n))

# 3. Hyperparam tres
alpha = 0.1         # Learning Rate (Vitesse d'apprentissage)
gamma = 0.99        # Discount Factor (Importance du futur)
epsilon = 1.0       # Exploration Rate (100% al atoire au d but)
epsilon_decay = 0.995 # On r duit l'exploration de 0.5%    chaque pisode
min_epsilon = 0.01

episodes = 2000

for episode in range(episodes):
    state, info = env.reset() # On remet l'agent au d part
    done = False

    while not done:
        # --- Strat gie Epsilon-Greedy ---
        if np.random.rand() < epsilon:
            action = env.action_space.sample() # Exploration (Hasard)
        else:
            action = np.argmax(q_table[state, :]) # Exploitation (Meilleur connu)

        # --- Interaction avec l'environnement ---
        next_state, reward, terminated, truncated, info = env.step(action)

```

```
        # --- Mise    jour Q-Learning (Formule Math matique) ---
        # On regarde quelle est la valeur MAX possible au prochain  tat
        max_next_q = np.max(q_table[next_state, :])

        # La valeur actuelle
        current_q = q_table[state, action]

        # Le calcul de l'erreur et la mise    jour
        # Q(s,a) = Q(s,a) + alpha * (Reward + gamma * max(Q(s')) - Q(s,a))
        new_q = current_q + alpha * (reward + gamma * max_next_q - current_q)

        q_table[state, action] = new_q

        # On avance
        state = next_state
        done = terminated or truncated

    #    la fin de l' pisode , on r duit epsilon
    epsilon = max(min_epsilon, epsilon * epsilon_decay)

print("Apprentissage termin  !")
print(q_table)
```